

**Multi-Agent Learning & Imperfect Information Games:  
Final Report Encompassing *Spring 2025* Semester**

Mauricio Garcia

Supervised by Dr. Tian

University of Texas - Austin

April 2025

# Introduction

Imperfect-information games, in which each participant holds private information and must infer or bluff about hidden details, pose unique challenges for AI. Traditional methods from perfect-information domains (for example, brute-force search in chess) cannot directly handle hidden information or deception. Instead, I adopted Counterfactual Regret Minimization (CFR), an iterative self-play algorithm that converges to Nash equilibrium strategies for extensive-form games. CFR has become fundamental in solving poker and other bluffing games, automatically revealing sophisticated tactics without explicit programming.

This report opens with a walkthrough of CFR on the classic Kuhn Poker toy game. After experimenting with everything from Rock Paper Scissors to Leduc and Kuhn Poker—and *many* of Liar's Dice versions—I decided to focus this report on one custom three-stage Liar's Dice game (bid → response → truth/lie reveal) as my imperfect-information testbed. I describe the design of that game, detail my CFR implementation (including state abstraction, action pruning, opponent-modeling tweaks, and a “blueprint + subgame resolving” approach inspired by Sandholm's Libratus & Pluribus system), and evaluate the resulting strategies through both self-play and matchups against fixed-strategy opponents.

Finally, I discovered that while CFR excels at finding unexploitable equilibrium strategies, it does not natively deviate from equilibrium to exploit human play patterns without extensive fine-tuning—a challenge I wasn't able to overcome in my custom Liar's Dice variant, given the difficulty and project's time constraints.

## Background: CFR and Kuhn Poker

**Counterfactual Regret Minimization (CFR)** is an iterative algorithm for finding approximate Nash equilibria in extensive-form games with imperfect information. The key idea is to have the players self-play the game repeatedly, adjusting their strategies based on regret for suboptimal choices in hindsight. After each iteration, if a player realizes that an alternative action would have yielded a better outcome given the opponents' play, the algorithm increases the propensity of that action (this is regret matching). An engine was made for this, which I'll go into more detail about later.

Over many iterations, the strategy frequencies converge such that no player has significant regret for any single deviation, meaning the strategy profile approximates a Nash equilibrium. A crucial result in CFR is that the average strategy converges to a Nash equilibrium as cumulative regret diminishes. This allows strong, unexploitable strategies to be learned even in games where explicit calculation of equilibrium is intractable. For instance, heads-up no-limit Texas Hold'em, the benchmark imperfect-information poker game, has on the order of  $10^{165}$  distinct game states once you account for all possible private cards and betting sequences. There's no hope of writing down and solving the exact Nash equilibrium for a game that size by traditional methods. Yet in 2017, Libratus—built on a CFR-based blueprint plus subgame resolving approach—computed an approximate equilibrium strategy for that same massive game and went on to defeat top human pros by a statistically overwhelming margin. That result shows that CFR can learn extremely strong, effectively unexploitable strategies, even when explicit equilibrium computation is mathematically and computationally intractable.

A toy example for CFR is **Kuhn Poker**, a simple two-player poker game introduced by Harold Kuhn. Kuhn poker uses a deck of just 3 cards (typically the Jack, Queen, King) where each player antes (small forced bet to enter a Poker game) 1 chip and receives one private card. There is a single round of betting: players can either pass (fold/check) or bet (wager an extra chip). If one player bets and the other passes, the bettor wins the pot immediately; if both players pass or both bet, they proceed to a showdown where the higher card wins the pot.

Despite its small size (only 3 cards leading to 12 information sets in total), Kuhn poker exhibits the essential features of poker: players must bluff with weak hands and sometimes slow-play (i.e. check) strong hands to maximize value. In fact, Kuhn poker was among the first games analyzed in game theory and was solved analytically by hand in 1950. Its equilibrium strategy is intuitive yet instructive; in equilibrium, when you hold the worst card, you'll bluff (i.e. bet) only with a small probability, and when you hold the best card, you'll bet almost every time. Those exact bluff-and-bet probabilities are chosen so that whenever your opponent faces a bet, their expected payoff from calling equals their payoff from folding. CFR can be applied to Kuhn poker as a didactic example (credit to Sandholm's Computational Game Solving course at CMU), iteratively converging to this equilibrium strategy.

Implementing CFR on Kuhn poker demonstrates how the algorithm automatically learns to bluff and call optimally: classic behaviors like bluffing arise naturally from the regret minimization dynamics. This small-scale success illustrates CFR's relevance to solving larger imperfect-information games: if it can solve Kuhn's mini-poker, then with enhancements it can tackle far more complex games like Texas Hold'em. Indeed, modern poker AIs use CFR at their core – the best poker programs are now generated via self-play algorithms based on game-theoretic principles, a leap from the early days when even large poker games were approached with rule-based systems. CFR provides a systematic way to approach optimal play, making it a natural choice for designing an AI for any game of hidden information and strategic bluffing.

## Game Design: A Custom 3-Stage Liar's Dice Variant

To explore CFR in a richer & personalized setting, we designed a custom game inspired by **Liar's Dice**, a classic dice bluffing game. In Liar's Dice, multiple players roll dice in secret and take turns claiming how many dice of a certain value appear under all cups, often exaggerating (bluffing) their claims. Others can either accept the claim or challenge it by

calling the bluff (“liar!”), at which point, dice are revealed to determine the truth. We created a simplified two-player, single-round variant of Liar’s Dice with three distinct stages: **Bid**, **Response**, and **Truth/Lie Reveal**. This 3-stage structure captures the essence of one cycle of bluffing and calling, while being self-contained enough for CFR analysis. The rules and structure of the game are as follows:

- **Private Dice Roll (Setup):**

- At the start, each player rolls a set of dice in secret. In our implementation, both Player 1 (P1) and Player 2 (P2) receive a private hand of dice (5 dice each with 6 faces). The results are known only to the respective player, modeling private information. This secret roll by “chance” sets up an information asymmetry: each player knows their own dice but not the opponent’s.

- **Stage 1 – Bid (P1’s Claim):**

- In the classic Liar’s Dice variant, each bid specifies both a quantity  $Q$  and a face value  $F$ , as in “There are at least 3 dice showing face 5.” This means “across all hidden dice, I claim you’ll find three 5’s.” In our simplified version, we drop the face-value detail and only care about the maximum frequency—that is, the count of whichever face appears most often. A bid now reads:

- “Max-frequency  $\geq Q$ ”

- When making a bid, P1 sees their own dice and chooses  $Q$  by weighing:

- Plausibility: A higher  $Q$  is less likely to be true (more of a bluff).

- Reward: If P2 doesn’t call, an unchallenged higher  $Q$  wins outright.

- We also prune any impossible bids, like a  $Q$  greater than the total number of dice or  $Q$  that P1’s hand plus the maximum the opponent could hold can’t reach. This keeps the decision set both small and logically consistent.

- **Stage 2 – Response (P2’s Challenge or Accept):**

- Once P1 announces “Max-frequency  $\geq Q$ ,” P2 has two choices:
  - **Raise:** Increase the bid to  $Q' > Q$ , forcing P1 back on the defensive.
  - **Call BS:** Challenge P1’s claim immediately.

- If P2's hand already contains close to Q of some face, then "Max-frequency  $\geq$  Q" is quite plausible, and P2 is likelier to raise (or even pass by raising minimally). If P2's hand makes Q seem out of reach, you can call a bluff. It can be summarized as follows: "Given my dice, how likely is it that some face reaches Q across both hands?"

● **Stage 3 – Truth/Lie Reveal (Resolution):**

- **If P2 called BS in Stage 2**, all dice are checked, and you check the actual maximum frequency vs. the last bid **Q**:
  - **Actual  $\geq$  Q** → P2 was wrong to call, so P2 loses a die.
  - **Actual  $<$  Q** → P1 was caught bluffing, so P1 loses a die.
- **If P2 raised in Stage 2**, P1 now must **declare** "truth" or "lie" about their own claim, then everyone reveals their dice, and you compare actual vs. Q:
  - P1 says "truth"
    - If actual  $\geq$  Q, P2 loses a die.
    - If actual  $<$  Q, P1 loses a die.
  - P1 says "lie"
    - If actual  $<$  Q, P2 loses a die.
    - If actual  $\geq$  Q, P1 loses a die.

This final stage ties together the bluff and the call/raise, rewarding correct calls and catches, and penalizing mistaken challenges or failed bluffs. This version keeps the game tree tractable for CFR analysis. I was not particularly content with the third stage, as it does not feel like a particularly 'fair' or 'balanced' ending. It is also not particularly intuitive or easy to understand. In future work, I'd like to build on a more intuitive game (like normal Liar's dice, etc.).

## CFR Training with a Unified Engine

The CFR logic is implemented once in `cfr_train.py` using a generic CFR class (from `stub.py`, obtained and developed from Sandholm) parameterized by `RegretMatchingPlus`. Both Player 1 and Player 2 policies are computed in the same loop. Here's the high-level workflow:

## 1. Game Setup and Tuplification

- Load the JSON-encoded extensive-form game (decision nodes, observation mappings, and utility entries).
- Convert list-based edges and sequences into tuples for dictionary keys.

## 2. CFR+ Iterations

- Instantiate two CFR solvers:
  1. `cfr1 = Cfr(dp1, rm_class=RegretMatchingPlus)`
  2. `cfr2 = Cfr(dp2, rm_class=RegretMatchingPlus)`
    1. where `dp1` and `dp2` are the decision-problem lists for P1 (bid/truth) and P2 (respond).
- Maintain cumulative sequence-form strategy sums (`cum1`, `cum2`) to build the average strategy over iterations.
- **Loop** for `t` in `1..N` iterations (for example, 50k which I used):
  1. **Next Strategy:** Call `cfr1.next_strategy()` and `cfr2.next_strategy()`, yielding realization-probability vectors. These are mappings from each decision sequence to the probability of reaching that sequence under the current mixed strategy.
  2. **Accumulate:** Add these (weighted by `t`) to `cum1/cum2` to form the running average.
  3. **Utility Computation:** Compute counterfactual utility vectors `u1` and `u2` with the opponent's current strategy.
  4. **Regret Update:** Call `cfr1.observe_utility(u1)` and `cfr2.observe_utility(u2)` to update cumulative regrets at each node.

### 3. Policy Extraction

After convergence, normalize the cumulative weights at each decision node to extract corresponding JSON policies.

### 4. Unified Engine

A single CFR implementation serves both players: the same **Cfr** + **RegretMatchingPlus** machinery handles P1's bid and truth/lie stages and P2's response stage. This consistency avoids code duplication and ensures identical update logic.

### 5. Abstractions & Pruning

- **Action Pruning**

- **State Abstraction:**

1. Treat dice hands as multisets to collapse permutations into single info-sets.

- **Scaling Experiments:**

1. I trained on different factors, such as different starting dice amounts.

### 6. Extensions & Live Resolving

- **Blueprint + Resolve:**

1. Maintain the offline equilibrium (via "blueprint") and, in live play, analytically compute  $P(\max \geq Q \mid P2's \text{ dice})$  for off-path bids.

- **Opponent Modeling:**

1. Freeze P2's strategy as "always-call" or "always-accept" (amongst others) to derive the best exploitative responses.

The end result is three JSON policies mapping each info-set to an equilibrium mixed strategy, ready for both large-scale self-play evaluation and interactive human-facing play.

**stub.py** provides a more general CFR engine (with both **RegretMatching** and **RegretMatchingPlus**) used in coursework to explore theoretical properties. It was from a



homework assignment in Sandholm's [course](#) to solve multiple problems given. `cfr_train.py` was fully developed outside of a course, and it uses RegretMatchingPlus CFR+ to compute full equilibrium policies offline, focusing on convergence speed and blueprint generation. It's more tailored for production, and not necessarily classroom problems. For context, RegretMatching accumulates all regrets (positive and negative), whereas RegretMatchingPlus (CFR+) clamps any negative regret to zero each update, focusing only on beneficial actions and speeding up convergence.

## Evaluating Performance and Scalability

### Convergence to Equilibrium

In the 3-stage version described, I managed to reach Nash equilibrium:

- Self-play win Rate  $\approx$  50 %

After 50,000 CFR+ iterations on the full three-stage, 5-dice-per-player variant, two independent AI instances split wins at 49–51 % regardless of who opened the bid. This symmetry shows neither side holds an exploitable edge, i.e., the average policy approximates a Nash equilibrium.

### Exploiting an Always-Call Opponent

```
=== 2-Stage Liar's Dice Exploit Demo (3 dice each) ===
```

```
--- Round 1 (AI: 3 dice, You: 3 dice) ---
```

```
Your roll: [6, 4, 1]
```

```
AI opens  $\geq$  1
```

```
You: call
```

```
AI's hand: [3, 2, 3] (true max-freq = 2)
```

```
→ You lose a die!
```

```
--- Round 2 (AI: 3 dice, You: 2 dice) ---
```

```
Your roll: [2, 5]
```

```
You open  $\geq$  2
```

AI: calls BS!  
AI's hand: [6, 4, 1] (true max-freq = 1)  
→ You lose a die!

--- Round 3 (AI: 3 dice, You: 1 die) ---  
Your roll: [4]  
AI opens  $\geq 1$   
You: call  
AI's hand: [6, 1, 5] (true max-freq = 1)  
→ You lose a die!

\*\*\* AI wins the match! \*\*\*

*This max-frequency style game follows the traditional two-round until the highest bid approach. All bids  $\geq 1$  are doomed against a caller; the AI finishes at a full 3-die advantage, playing a flawless game.*

## Vulnerability to Deviations

Here we see that a seemingly innocuous deviation from the equilibrium “always-call” benchmark—namely, always raising from  $Q=1$  to  $Q=2$ —causes the AI’s blueprint to fail. In the five-round “deviation demo,” the AI loses every die in succession, wiping out its 3-die starting advantage in a single match:

=== Deviation Demo: opponent always raises 1→2 instead of call ===

R1: AI=[4, 1, 3] You=[5, 5, 4] bid $\geq 2$  → You lose 1 die  
R2: AI=[3, 1, 1] You=[4, 3] bid $\geq 2$  → You lose 1 die  
R3: AI=[1, 5, 3] You=[4] bid $\geq 2$  → AI loses 1 die  
R4: AI=[6, 5] You=[3] bid $\geq 2$  → AI loses 1 die  
R5: AI=[4] You=[1] bid $\geq 2$  → AI loses 1 die

After deviation: AI has 0 dice, You have 1 dice.

*This reversal highlights a key tension in imperfect-information AI: equilibrium strategies can be catastrophically fragile when the opponent strays from the assumed distribution. In practice,*

*then, pure CFR must be augmented with opponent-modeling or live subgame solving to remain robust in the wild.*

## Computational Performance

Using max-frequency (i.e. collapsing each hand to the single number representing the highest count of any face) dramatically simplifies the game's state space:

- Traditional face-specific Liar's Dice:
  - Each player's 5-die hand is a histogram over faces 1–6 (e.g. 0 ones, 2 twos, 1 three, 0 fours, 1 five, 1 six). There are
  - $\binom{5 + 6 - 1}{6 - 1} = \binom{10}{5} = 252$  possible frequency profiles (how many dice show each number).
  - *5 is the number of dice you roll, 6 is the number of possible face values, and 1 comes from the standard combinations-with-repetition formula.*
  - Across both players, that's ( $252^2 \approx 63,500$ ) distinct chance-node outcomes before any bidding occurs
- Our custom max-frequency adaptation:
  - We reduce each hand to a single integer ( $m \in \{1, 2, 3, 4, 5\}$ ) (the count of the most common face). That leaves just 5 possibilities per player, or ( $5^2 = 25$ ) combined chance outcomes. This simple adaptation makes the game much more manageable and malleable.

In effect, max-frequency collapses the raw game tree by a factor of over 2,500× at the chance-node level alone, even before pruning bids or responses. Downstream, the number of information sets across all three stages shrinks from on the order of >100,000 (in a face-specific model) to <≈12,000 with max-frequency abstraction.

## Practicality

From a practical standpoint, max-frequency abstraction yields substantial efficiency gains without specialized hardware. By collapsing each hand to a single integer, the memory footprint for storing regrets and cumulative strategies shrinks from on the order of hundreds of thousands of entries ( $\gg 1$  GB) down to just a few thousand ( $\ll 1$  GB). This leaner state space also accelerates each CFR+ iteration by what seems roughly 30–40 %, which in turn reduces the total number of iterations required to reach an average regret of  $\leq 0.01$  by a similar margin. On

a MacBook Pro M2 Pro with 16 GB of RAM, running 50,000 CFR+ iterations on the full three-stage, 5-die max-frequency variant completes within a reasonable window.

By abstracting, it is also easier to use interactive experimentation and develop UIs, as you do not have to track the faces of the dice. However, one of the trade-offs of this abstraction is clear: while we gain dramatic tractability and a straightforward policy representation, we forgo the ability to exploit opponents whose bluffing behavior hinges on specific face distributions. Nevertheless, by preserving the core dynamics of bluffing and calling, max-frequency abstraction strikes a practical balance, enabling efficient CFR+ training on consumer-grade hardware.

## Key Lessons Learned

- Equilibrium  $\neq$  Exploitation (High Win-rate):
  - CFR+ converges to strategies with zero regret, but it does not automatically exploit suboptimal or non-equilibrium opponents. Without dedicated opponent modeling, the win rate will never be particularly high.
- Blueprint + Subgame Resolving:
  - Storing an offline “blueprint” equilibrium and selectively resolving future decision points can adapt to opponent behavior and patch the brittleness seen in the deviation demo.
- Abstraction Trade-offs:
  - Collapsing dice permutations into multisets yields massive state-space reductions, but over-abstraction risks throwing away exploitable patterns. Finding the right granularity is crucial.
- CFR+ Speeds Convergence:
  - Clamping negative regrets to zero (using RegretMatchingPlus) accelerates convergence in large info-sets, making CFR practical on a laptop for medium-sized games.

## Conclusion

Over the course of this project, I've gained a deep appreciation for the unique challenges posed by imperfect-information games and the power of Counterfactual Regret Minimization to tackle them. I learned about a problem I hadn't really thought through before—imperfect information—and now I feel equipped to handle these scenarios a lot better going forward. Designing and implementing a custom three-stage Liar's Dice variant

showed me how a simple max-frequency abstraction can yield dramatic efficiency gains, while also highlighting the trade-offs between tractability and exploitability. The biggest lesson was most definitely that equilibrium strategies often stumble against real human opponents unless you go the extra mile, like layering in live subgame resolving or solid opponent modeling. It's not as simple as training a Q-learning agent and walking away—you really have to lead it in the right direction. Armed with these lessons, I'm excited for the future; some work that can be done: refining my game design, deepening opponent models, and deploying a lightweight UI so that I can test against human players and iterate even faster.

## References

- Brown, N., & Sandholm, T. (2018). Superhuman AI for heads-up non-limit poker: Libratus overcomes human talent. *Science*, 359(6374), 418–424.  
<https://doi.org/10.1126/science.aao1733>
- Brown, N., & Sandholm, T. (2019). Pluribus: The first AI to beat pros in heads-up, no-limit Texas hold 'em poker. *Science*, 365(6456), 885–890.  
<https://doi.org/10.1126/science.aay2400>
- [Superhuman AI for heads-up no-limit poker: Libratus beats top professionals](#)
- Kuhn, H. W. (1950). A simplified two-person poker. In H. W. Kuhn & A. W. Tucker (Eds.), *Contributions to the Theory of Games I* (pp. 97–103). Princeton University Press.
- Neller, T. W., & Lanctot, M. (2012). Counterfactual Regret Minimization: A tutorial. Technical Report, McGill University, Canada. Retrieved from  
<https://www.cs.toronto.edu/~vmnih/docs/draft-chapter.pdf>
- Sandholm, T. (2021). Computational Game Theory and Equilibrium Finding (CMU CS 15-888 lecture notes). Carnegie Mellon University. Retrieved from  
<https://www.cs.cmu.edu/~sandholm/course/15-888-F21/lectures.html>
- [https://www.researchgate.net/publication/235737910\\_Measuring\\_the\\_Size\\_of\\_Large\\_No-Limit\\_Poker\\_Games](https://www.researchgate.net/publication/235737910_Measuring_the_Size_of_Large_No-Limit_Poker_Games)
- <https://thegradient.pub/libratus-poker/>

- [https://www.researchgate.net/publication/221605456\\_Effective\\_Short-Term\\_Opponent\\_Exploitation\\_in\\_Simplified\\_Poker](https://www.researchgate.net/publication/221605456_Effective_Short-Term_Opponent_Exploitation_in_Simplified_Poker)